

Projektzusammenfassung & Statusbericht

Zustand, Tech-Stack, erreichte Meilensteine und zukünftige Roadmap

Datum: 18. Mai 2026 | Entwickler: Ed / Webstark.org | Projekt-Modus: Local & Cloud Jarvis Edition

1. Projektübersicht

Der **Elite Desktop Agent (Jarvis Edition)** ist ein hochmoderner, modularer KI-Assistent für Windows. Das System agiert als diskreter Butler im Hintergrund, der über Echtzeit-Spracherkennung verfügt, proaktiv handelt und die vollständige Kontrolle über die Windows-Umgebung besitzt (Tastatur, Maus, Shell, OCR). Ein futuristisches HUD-Design im Glassmorphismus-Stil bildet die Benutzeroberfläche und visualisiert die KI-Gedankengänge (fury-sdk), Systemmetriken, Kamerabilder und Automationen in Echtzeit.

2. Technologie-Stack

Komponente	Technologien & Frameworks
Frontend (HUD Interface)	Next.js, LiveKit Components (React), Tailwind CSS, Framer Motion (futuristische Spring-Animationen & Theme-Stile)
Desktop Container	Electron Wrapper (main.js, services.js) zur nahtlosen Desktop-Integration, Hotkeys, Tray-Management und automatischen Service-Orchestrierung
Backend Agent (Jarvis Core)	Python 3.10+, LiveKit Agents SDK, OpenAI Realtime API (ash voice / VAD-Integration) für native Sprachinteraktion in Echtzeit
Vision & Frame Processing	OpenCV, Pillow & Base64-Streaming für Kognitive Wahrnehmung (GPT-4o Vision Webcam-Feed & Desktop-OCR)
System-Automation (Action)	PyAutoGUI, PSUtil und native Windows UI Automation über eine spezialisierte C# GPRC-Schnittstelle (UiAutomationGRPC.Server.exe)
Mission Control Hub	Node.js (Express), SQLite-Datenbank zur permanenten Aufgabenverwaltung, GitHub Issues Sync & Webhook Events
Bereitstellung & Build	Microsoft winapp CLI, Custom MSIX Packaging Pipeline mit Entwickler-Code-Signing (devcert.pfx), Startup-Batches

3. Aktueller Projektstatus & Fortschritte

Das System wurde in den letzten Zyklen erfolgreich stabilisiert und für den professionellen Einsatz optimiert. Folgende Meilensteine wurden erreicht:

- **Modulares 8-Widget Dashboard (/dashboard):** Integration eines vollwertigen HUD-Panels bestehend aus System Monitor (Echtzeit CPU/RAM/Disk), Webcam Feed (Scans), Log Streamer (Echtzeit-Denkprozesse), Music Controller (Spotify & Audio-Visualizer), Image Archive (Vision-Bilder), AI Chat, Smart Clipboard Text Editor und Command Index.
- **Asthetik-Feinschliff:** Anpassung des Neon-Cyan Glassmorphismus-Themas mit optimierten, dezenten Rändern und Framer-Motion Widget-Animationen für einen extrem hochwertigen Eindruck.
- **Vollautarker Lokaler Modus (Docker & LiveKit-Server):** Implementierung einer automatischen Erkennung und Steuerung des lokalen LiveKit-Dienstes. Docker Desktop wird bei Bedarf via PowerShell gestartet, und der Docker-Container livekit-server wird vollautomatisch mit 0.0.0.0-Bindung und UDP WebRTC-Portweiterleitung (7882) neu erstellt, um Loopback-Verbindungsfehler zu verhindern.
- **Native Windows MSIX Deployment-Pipeline:** Erfolgreiches Setup der Build-Pipeline (pnpm run build:msix), welche Next.js Standalone-Assets, Mission Control und den Desktop-Agenten in ein signiertes Windows-MSIX-Installationspaket packt.
- **Dateisystem-Schutz & Rechte-Stabilisierung:** Behebung kritischer Schreibberechtigungs-Konflikte (EPERM), die durch die restriktive WindowsApp-Ordnerstruktur des MSIX-Pakets auftraten. Stateful Konfigurationen (wie config.json und Heartbeats) wurden vollständig in beschreibbare Benutzer-AppData-Verzeichnisse ausgelagert.
- **Voice-Core v2 & Rauschreduktion:** Optimierung des Sprachmoduls durch Ultra-Strict VAD (Voice Activity Detection Threshold 0.8 / 0.4) und eine beschleunigte Mikrofon-Initialisierung (1.5 Sekunden) zur Minimierung von Hintergrundrauschen.
- **Integrierte Assistenz-Feature Sets:** Volle Funktionsfähigkeit von Workflows wie dem **Ghost Mode** (sofortiger Sichtschutz), **Meeting Guard** (Auto-Mute von Spotify und Öffnen des Notizblocks bei Calls) und dem **Smart Clipboard Monitor**.

4. Zukünftige Entwicklung & Roadmap

Die kommenden Phasen konzentrieren sich auf eine tiefere Hardware-Integration und die Erweiterung der Mission Control Automatisierungsfunktionen:

Phase A: Physische & Hardware Integration

- **ESP32 / IoT Smart Home Bridge:** Direkte Steuerung von Hardware-Modulen (z.B. Lilygo T-RGB, T-Display-S3 oder Smart Home Lampen) aus dem Jarvis Core per MQTT-Protokoll oder HTTP-Requests.
- **Vision-Based Webcam-Sicherheit ('Stranger Alert'):** Proaktive Webcam-Überwachung, die beim Erkennen unbefugter Personen automatisch reagiert (z.B. den Bildschirm sperrt, Fenster minimiert oder eine Benachrichtigung sendet).

Phase B: Mission Control Erweiterungen (Dashboard)

- **GitHub Issues Integration:** Vollständige Aktivierung der Issues-Synchronisation über GitHub Personal Access Tokens (Classic) in `mission-control/.env`, um GitHub Issues direkt als Aufgaben in der Dashboard-Inbox verwalten zu können.
- **Agent Profile & Memory Mirroring:** Synchronisations-Skript, welches die lokalen Profildateien des Agenten (`GEMINI.md`, `.agent/CONVERSATION_MEMORY.md`) automatisch in das Mission-Control-Verzeichnis (`.mission-control/agents/elite-agent/`) spiegelt.
- **Webhooks & Notification Systems:** Einrichtung von aktiven Webhooks im Verzeichnis `.mission-control/hooks/` zur automatischen Statusübermittlung an externe Chatdienste (z.B. Discord oder Telegram), sobald Aufgaben erledigt sind.
- **Claude Terminals Connection & Log Spiegung:** WebSocket-Kopplung im Python-Backend an die Mission Control API zur Echtzeit-Spiegelung aller CLI-Sitzungen und internen KI-Gedankengänge im Terminal-Widget des Dashboards.

Ende des Berichts. Dieser Bericht spiegelt den Stand vom 18. Mai 2026 wider. Alle Kernsysteme laufen stabil, Docker-Automatisierungen sind einsatzbereit, und die MSIX-Verteilungspipeline ist vollständig funktionstüchtig.